Extended Abstract

Motivation In recent years, the application of LLMs has shown remarkable results in various domains. In a sort of meta-outcome, we have used human preference data to train models, but today these same LLMs are able to judge the quality of tasks performed by others, humans or LLMs.

In this project, we originally started training a lightweight model on a simple dataset. We took this as motivation to explore if the use of LLMs as judge, can both help improve outcomes as well as gauge the effectiveness of our training task itself.

Method We chose to use two models, GPT 4o and Gemini 1.5 Flash, as our judges.

Our judges got evaluated and compared the inference responses of 5 candidate LLMs:

- **SFT Baseline**: The SFT trained Qwen2.5 0.5B Base model we completed in a pre-requisite part of this project.
- **DPO Baseline**: The DPO trained model we completed in a pre-requisite part of this project.
- **Qwen Instruct**: The Instruct equivalent of the Qwen2.5 0.5B Base model.
- **GPT 3.5 Turbo**: The GPT 3.5 Turbo model.
- **Gemma 2b**: Google's Gemma 2B model.

Implementation All our participant LLMs ran inference for the same 400 prompts, as provided in the held out set of prompts for our project leaderboard. Our judges picked the best answer and rationale. We kept score of how many times each candidate was picked as the winner.

Results We know which judges picked which candidates for each of the 400 prompts. We also submitted the judge-curated responses to the leaderboard. However we do not yet know if one judge was significantly better than the other, while we wait for Leaderboard results to publish.

Discussion We see that despite eleven hours of SFT training our Qwen Base model has some ways to go to really compete with the Qwen Instruct model, but it is surprisingly in the same league as the Gemma1.1 model. Moreover the GPT3.5 model is preferred an order of magnitude more times than the Qwen Instruct model.

Both our judges appear to perform similarly. Thus validating each others results, but also showing us that they both have similar room for discernment among the candidate models. Perhaps even more capable models could be used to probe differences between our candidates more carefully.

Conclusion We are at a frontier in LLMs interacting with each other, whether it be through synthetic data generation, through multi-agentic teamwork using different models in a crew, or through tool use. Having LLMs become capable of training to become better than their past selves, as well as being able to discern each other's abilities, opens the door to LLMs being able to help each other improve.

Improve Instruction Following by using LLM as a judge

Rohit Makhija
CGOE, Stanford
rohitm@stanford.edu

Abstract

We explore LLM-as-a-judge framework, to improve instruction following while also measuring how effective a model training project has been, and whether two LLM judges are able to discern the quality of the candidates any differently.

But we also see that LLMs are now used for the very tasks needed to create them, or improve them. LLMs are now able to generate synthetic data, perform tool use, and even train other LLMs. We explore one of these meta-applications, where we use LLMs as judges to evaluate other LLMs. In truth, we are evaluating both the candidates, and the judges. And in our setup, we are also evaluating if our candidates have improved themselves through training.

While the results are not surprising, we are able to see a relative scale of difference between our candidates, and a marked similarity between our judges. This reveals a lot about our candidates and our judges, and gives us a framework we could extend to benchmark other models.

More than just benchmarking, this approach gives us a framework to evaluate the improvement of a model through training, and to evaluate the effectiveness of an LLM judge in discerning the work of less capable models.

1 Introduction

LLMs have rapidly become a ubiquitous tool in recent years. Their use spans domains such as personal assistants, tutoring systems and automated content generation. Central to this success is ability to follow instructions. Aligning the outcome of an LLM's response to a given instruction is fundamental to their success.

Traditionally LLMs have been trained and such alignment has been attempted through Reinforcement Learning from Human Feedback (RLHF). This requires human actors to take action. They might compare model outputs and provide explicit preference feedback. This is expensive, slow and difficult to scale.

Recent developments have revealed promising alternatives. We can use LLMs themselves to do some of these tasks that previously could only be done by humans. Whether it is data generation, tool use or even evaluating preferences, LLMs are able to learn and generalize enough to do these tasks. Extending the ability to learn and evaluate preferences, LLMs can now act as an evaluator or a judge.

This opens up a lot of possibilities for training LLMs to evaluate each other's work and creatively utilize a signal for how well an LLM is performing a task. One such application is to use LLMs as judges to evaluate other LLMs.

This project explores the use of LLMs as judges to evaluate other LLMs, and draw insights about both the candidates that are judged, as well as the judges that do the evaluations. Specifically we explore whether our judges are able to discern the quality of the candidates any differently and if the difference is significant.

The objectives of this project are to explore how closely different judges evaluate the same candidates, and how different the performance of different candidates turns out to be, and how we can utilize this to evaluate and improve LLMs during training.

2 Related Work

There is growing interest in using LLMs as a judge to evaluate the quality of other tasks. Prominently, Gu et al. (2025) survey the space and speaks to terminology, methods and applications. Meanwhile, Dorner et al. (2025) explore when LLM as a judge won't beat twice the data.

Finally Tan et al. (2025) introduces JudgeBench, a benchmark for evaluating LLM-based judges.

Inspired by these works, we explore the use of this newly popular concept of LLM as a judge to both qualitatively improve instruction, and quantitatively gauge how effective a model training project has been when working with a simpler model and benchmarking it against a more complex model.

3 Method

Our methodology aims to evaluate and identify opportunities to improve instruction following capabilities of a lightweight LLM using preference signals generated by larger LLMs acting as judges. We describe the architecture of our system in four stages: model selection and fine tuning, dataset preparation, evaluation protocol using LLM judges, and preference aggregation and analysis.

3.1 Models, Datasets and Fine Tuning

We adopt the `Qwen-2.5-0.5B Base` model as our baseline. It is a compact, general purpose transformer model with apprximately 500million parameters. Our initial model is fine-tuned using the SmolTalk dataset, which has approximately 460,000 samples. This baseline is referred to Qwen SFT in this paper. During SFT, our objective is to maximize the likelihood of the desired response given the instruction prompt.

$$max_\theta \mathbf{E}_{x,y\in D} \sum_{t=1}^{|y|} \log \pi_\theta(y_t|x, y_{<t})$$

We further perform Direct Preference Optimization on our SFT baseline model, using the UltraFeedback dataset, which has approximately 61,000 samples. This results in our Qwen DPO model. During DPO, our objective is to maximize the difference between the likelihood of the preferred response versus the rejected response, relative to a frozen model and given the instruction prompt.

$$\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = \mathbb{E}_{(x,y_w,y_l)\sim D}[\log \sigma\big(\beta \cdot (\log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)})\big)]$$

Note, this can also be written as:

$$\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = \mathbb{E}_{(x,y_w,y_l)\sim D}[\log \sigma\big(\beta \cdot (\log \frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)} - \log \frac{\pi_{ref}(y_w|x)}{\pi_{ref}(y_l|x)})\big)]$$

The β term controls how aggressive the DPO is. A smaller β means we learn slowly and don't drift too far from the frozen reference model versus a higher β encourages faster learning. As a result, we do not need to be concerned about the need for regularization or KL divergence and we control just the β term. Additionally, the π_{ref} fraction can be precomputed and cached. We didn't reach this optimization in our implementation, but it would potentially allow us to use larger batch sizes at each iteration for the same GPU memory capacity, and scale well for multiple epochs of training since we could cache the frozen policy logprobs ahead of time.

3.2 Candidate Models

To evaluate relative performance, we include five candidate models:

- **SFT Baseline:** Our Qwen2.5-0.5B model trained with SFT on SmolTalk.
- **DPO Baseline:** Our Qwen2.5-0.5B model further trained with DPO on UltraFeedback.
- **Qwen Instruct:** A publicly available instruction-tuned version of Qwen2.5-0.5B.
- **Gemma 2B:** A 2-billion parameter open-source model released by Google.
- **GPT-3.5 Turbo:** A commercial general-purpose model known for its high-quality instruction following with a 175B parameter count.

3.3 Evaluation Protocol Using LLM-as-a-Judge

To evaluate the instruction-following quality of each model, we use two large LLMs as judges:

- **GPT-4o** (OpenAI, 2024)
- **Gemini 1.5 Flash** (Google, 2024)

Each judge independently evaluates outputs on a previously frozen and held-out set of 400 prompts. For each prompt x, all five models generate responses $\{y_1, y_2, \ldots, y_5\}$. We perform a comparison of all responses, prompting the judge to select the preferred response and provide a rationale.

We shuffle the order of the candidate responses for each judging event, so our judges could not learn to always prefer the same winning candidate based on the order of the candidates. After the judging event we unshuffle the responses to get the original order and identify the winner.

Judge Prompt Format: We use structured prompts of the form:

> `User prompt:` x
> `Response A:` y_a
> `Response B:` y_b
> `Response C:` y_c
> `Response D:` y_d
> `Response E:` y_e
> Analyze the quality of each response, and select the best one. Respond only with one letter (e.g., "A", "B", "C", or "D") and a short explanation.

Since GPT-3.5-Turbo accepts a system prompt, we also include a system prompt "You are a fair and expert judge of language model outputs.". Gemini 1.5 Flash does not accept such a system prompt.

The judges respond with a selection (A or B or . . .) and an explanation, which we record.

3.4 Assumptions

Our method is based on several key assumptions:

1. **Judge Reliability:** GPT-4o and Gemini 1.5 Flash produce preference judgments that correlate strongly with human preferences.
2. **Independence:** Judge decisions are independent of each other and not biased toward any specific model.
3. **Instruction Clarity:** Prompts are unambigous and specify the desired outcome clearly, allowing for consistent evaluation.

4 Experimental Setup

All candidates were queried with the same 400 prompts, using temperature 0.6. All judges were queried with temperature 0.0 to avoid probabilistic behavior.

We shuffled the order of the candidate responses for each judging event, so our judges could not learn and become biased toward one or the other candidate.

We used VLLM with SamplingParams(temperature=0.6, top_p=0.95, top_k=20) to run inference from locally saved copies of our own SFT and DPO trained models. We used VLLM again to optimize inference from Gemma 1.1. For Qwen Instruct, GPT 3.5 Turbo, GPT 4o and Gemini 1.5 Flash, we queried the models directly via their respectiv public APIs.

5 Results

5.1 Quantitative Evaluation

Table 1: Winning Candidate Frequency by Judge

Model	GPT-4o Judge		Gemini 1.5 Flash Judge	
	Count	%	Count	%
Qwen2.5 0.5B Base + SFT	3	0.75%	3	0.75%
Qwen2.5 0.5B Base + SFT + DPO	0	0.00%	0	0.00%
Qwen2.5 0.5B Instruct	88	22.00%	136	34.00%
Gemma 1.1	7	1.75%	8	2.00%
GPT 3.5 Turbo	302	75.50%	253	63.25%
Total	400	100.00%	400	100.00%

- Not surprisingly, our judges are able to pick a commercially successful model like GPT 3.5 Turbo as the most frequently chosen candidate for providing the best answer in most cases.
- Our own DPO trained model performs so poorly, that it could not even be picked over our SFT trained model. This points to both the difficulty of the DPO task, in training hyperparameter selection and potentially implicates issues in the implementation itself.
- Somewhat surprisingly, the Qwen Instruct model is preferred more times than even the commercial grade Gemma 1.1 model. This is likely due to the fact that the Gemma 1.1 model is very lightweight, and at a 2B parameter count, it is not able to meaningfully reason and learn to compete with QwenInstruct.
- Most refreshingly, our own SFT trained model is able to win a few rounds, and consistently with both judges.
- Between the two judges, we see a marked similarity in the results. We do see that generally a GPT judge favors a GPT candidate by a small margin and a Gemini judge favors a Gemini candidate by a small margin.

5.2 Qualitative Analysis

We investigate cases where our judges picked our Qwen SFT model. We saw Qwen SFT win 3 times with both judges. It appears the judges are not identical in their choice and pick Qwen SFT model in different examples. The winning characteristic appears to be short or very to the point responses, like True or False, or one sentence replies. In these scenarios, other models were sometimes more verbose. This led to the judges picking the Qwen SFT model. However the difference in quality in those cases was not very significant. Some examples below.

Example 230 Judged by Gemini 1.5 Flash in favor of Qwen SFT:

- **Prompt**: Input: Consider Input: O can é o mellor amigo do home. Output: Il cane è il migliore amico dell'uomo.
- **Qwen SFT**: Output: Il cane è il migliore amico dell'uomo. Output
- **Qwen DPO:** Output: Consider Input: O can be """ # O can é o mellor amigo do home . . . (continues with unintelligible response)
- **Qwen Instruct**: Il cane è il migliore amico dell'uomo.
- **Gemma 1.1**: Explanation: The sentence is not grammatically correct. The correct sentence should be

- **GPT 3.5 Turbo**: The input text "O can é o mellor amigo do home" is in Galician language. The translation to English is "The dog is man's best friend."

Example 337 Judged by GPT 4o in favor of Qwen SFT:

- **Prompt**: Identify the language of the following sentence: "Ọmọ Yorùbá ni mi, mo n sọ èdè Yorùbá." Answer with "Yoruba" or "English". PROBLEM: Ọmọ Yorùbá ni mi, mo n sọ èdè Yorùbá. SOLUTION:
- **Qwen SFT**: Yoruba The problem is asking for the language of a given sentence.
- **Qwen DPO:** The language of the Question: Is it possible to... Answer: 0. Question: Que ... (continues with unintelligible response)
- **Qwen Instruct**: Yoruba
- **Gemma 1.1**: Yoruba. The sentence is in Yoruba, a language spoken in West Africa
- **GPT 3.5 Turbo**: Yoruba

We examine where Gemma was chosen as the winner by our judges. Again we notice the qualitative difference is not significant among the candidates.

Example 11 judged by GPT 4o in favor of Gemma 1.1:

- **Prompt**: Identify the person's name in the following sentence: "During the conference, Dr. Emily Carter presented groundbreaking research on renewable energy."
- **Qwen SFT**: The person's name is Dr. Emily Carter. She was a professor of environmental
- **Qwen DPO:** The person's name is Emily Carter. The sentence is: ... (continues unnecessary response)
- **Qwen Instruct**: The person's name in the given sentence is Dr. Emily Carter.
- **Gemma 1.1**: The person's name is Dr. Emily Carter.
- **GPT 3.5 Turbo**: The person's name in the sentence is Dr. Emily Carter.

We note here that all our DPO responses are unusable. We see that the model was responding well at some checkpoints during training, but it would cycle through episodes of unintelligible or repeating responses before recovering completely and repeating the process. The last snapshot we have of our DPO model is unfortunately unusable. So it is in line with expectations that our DPO model is not able to perform well.

6 Prior work

We completed two pre-requisites to the project, Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO)

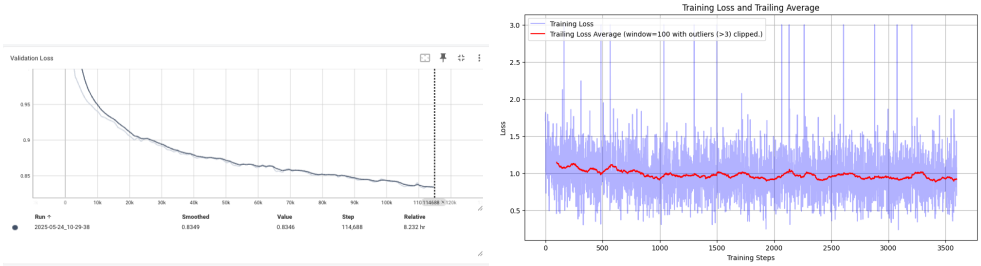
We use the Qwen2.5 0.5B Base model as our baseline model. We use the SmolTalk and UltraFeedback datasets, which have approximately 460,000 and 61,000 samples respectively.

6.1 Supervised Fine-Tuning

We use SmolTalk to perform Supervised Fine-Tuning (SFT) on the Qwen2.5 0.5B Base model. With mixed precision, gradient accumulation, and other optimizations, we were able to train the model for 1 epoch in 11 hours on an AWS g6e.xlarge instance. The resulting model became our new baseline model going forward.

We use preference feedback available in UltraFeedback to perform Direct Preference Optimization (DPO) on our SFT baseline model.

We successfully completed SFT on the base model over 1 epoch of all 460k samples in the SmolTalk training dataset, using batch size of 4. Execution with gradient accumulation, and mixed precision, took approximately 11 hours on AWS g6e.xlarge instance. We see a steadily decreasing Validation Loss, and a very gradually decreasing Training Loss.



We could have used the apply_chat_template library to tokenize the dataset, but we found that the tokenization was not consistent, and the model was not able to learn to tokenize the dataset correctly. So we used manual tokenization. This worked well and our model produced reasonable English responses, scoring a 0.49 on the leaderboard.

Leaderboard Submission: LateNightReinforcer_instruction_following_1748596403

6.2 Direct Preference Optimization

We performed DPO on the SFT baseline model, over 1 epoch of the full UltraFeedback training dataset. With a batch size of 3, gradient accumulation and mixed precision, this took approximately 5 hours on an AWS g6e.xlarge instance.

Our initial results suggested that either the loss function is not well implemented, or the dataset has a lot of variance in preferences. See figure 1

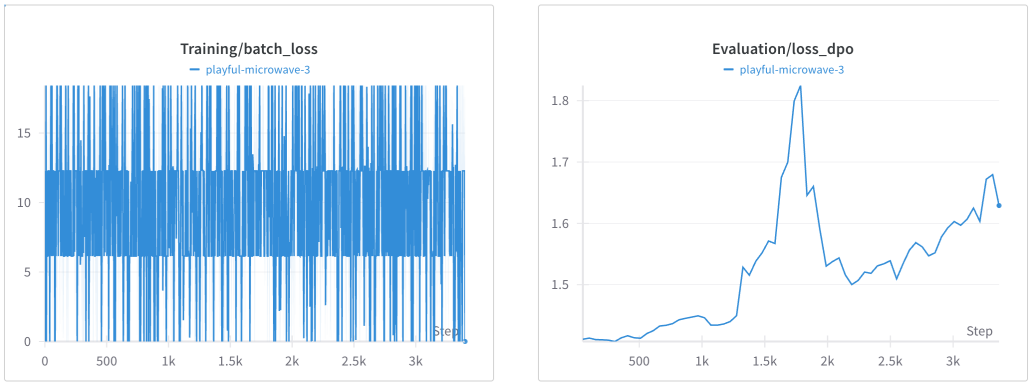


Figure 1: Initial DPO results showing potential issues with loss function implementation or dataset variance.

On further investigation, we found the loss function was not well implemented. Addressing this resulted in a satisfactory loss curve, and gradually improving accuracy of chosen versus preferred responses in the dataset, relative to the frozen baseline model. See figures 2 3

Despite those results, our DPO performance was not consistent. We see eval loss does not converge, and token accuracy is not consistently improving. See figure4

Given the premise that this is a solvable problem we hypothesize that there is a mix of two issues here.

1. Hyperparameters are not well tuned.
2. There are inconsistencies in tokenization, either due to our use of manual tokenization instead of the apply_chat_template library, or in Qwen's ability to learn and handle tokenization upon such repeated training.

Despite the numerous combinations we tried, we were not able to get consistent text responses from our last checkpoint on DPO. After every few batches during training, the evaluation examples being logged would appear to be gibberish, or have repetition. A few gradient steps later, the responses

Figure 2: Loss curve during DPO training showing satisfactory convergence.

Figure 3: Accuracy of chosen versus preferred responses in the dataset during DPO training.

would be reasonable English responses again. Sometimes we observed the model learn to make shorter responses, then other times it had qualitiatively worse responses. But all through this training, the training loss curve was consistently decreasing and the evaluation loss curve was not converging. We conclude that this kind of learning is a brittle problem and our model may yet recover with further training or further hyperparameter tuning.

Example:

- **Prompt**: Describe how the sea looks like when you are standing on the beach
- **Good Responses:**
  - The sea looks like a deep blue color with a lot of clouds.
  - Standing on the beach, the sea looks like a vast, blue-green expanse of water with a few small rocks and seashells in the distance.
  - When you are standing on the beach, the sea looks like a vast, blue and green ocean.

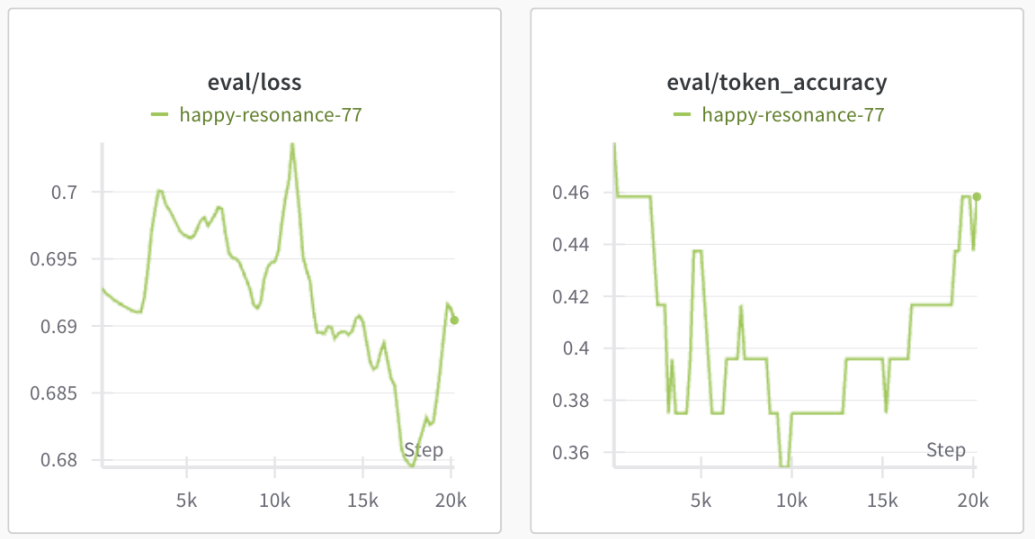


Figure 4: Evaluation metrics showing inconsistent convergence in DPO training, with non-converging eval loss and inconsistent token accuracy improvements.

- **Bad Responses:**
  - The sea looks like a sea of dark grey and green when you<|im_start|>assistant<|im_start|>assistant<|im_start|>assistant<|im_start|>assistant<|im_start|>assistant . . . (contd)
  - Qualitatively bad response: "I'm sorry, but as a text-based AI, I don't have the ability to experience physical sensations such as seeing or feeling. I am designed to provide information and answer questions based on the text you provide me with."

6.3 Leaderboard Submission

At the time of this writing, the leaderboard has not yet published a score for several of our submissions. We anticipate it will do poorly on the DPO benchmark just given where our DPO checkpoint was in its cyclic nature of underperforming and recovering, but hope to see promising results on the Extension benchmark.

- **SFT Submission**: LateNightReinforcer_instruction_following_1748596403. Score 0.49
- **DPO Submission**: LateNightReinforcer_instruction_following_1749525744.
- **Extension: GPT as Judge**: LateNightReinforcer_instruction_following_1749525669.
- **Extension: Gemini as Judge**: LateNightReinforcer_instruction_following_1749527765.

7 Discussion

7.1 Judge Bias and Model Preferences

We see some potential bias where a GPT judge favors a GPT candidate by a small margin, and a Gemini judge favors a Gemini candidate by a small margin. There is insufficient data in this experiment to draw strong conclusions. One potential hypothesis here is, that models from the same provider are trained and designed with similar biases, by similar teams, using similar data and assumptions about what answers are preferred versus not preferred. This is a hypothesis that could be tested at scale given more time and specific focus in the future.

7.2 DPO Training Challenges

With the qualitative comparison discussed earlier, we might conclude that picking the best answer was quite subjective. This is true when the inference responses are not very different in quality. But it is not true when one of the models performs significantly worse than the others, for example our DPO implementation. We noted earlier that our training loop exhibited a cyclic behavior of learning good and bad responses over a large number of training iterations within even the single epoch of training we did. Our model checkpoint was captured at a point where the model was not offering very usable responses. But the results of our LLM Judges are clear, they do not value the poorly formed responses from our DPO model relative to the other candidates, for any of the 400 prompts tested.

One potential outcome of this observation is that the LLM Judges could serve as a training signal. They could offer an early stopping signal when errors are aggregious, or at least a monitoring signal for trends to watch out for. Specifically for our DPO model, since we measure the loss of the model against a frozen copy of its own previous SFT trained state, its challenging to know if we're learning and reinforcing good behaviors or bad behaviors. We are heavily dependent on both the preference data we train on and our frozen model checkpoint. LLM Judges could offer us a neutral third party insight into preference data and help us learn with more confidence, for example by adding a multiplying boost to our β value in the DPO algorithm, or as discussed earlier, just act as a monitoring or early stopping signal.

7.3 Model Size and Performance

Somewhat surprisingly, Gemma performs significantly worse than Qwen Instruct. We acknowledge that Gemma 1.1 might be a lighter weight model than Qwen Instruct with 2B vs 2.5B parameters. But we didn't anticipate the difference in their performance to be so significant, given one is 20% more parameters than the other, not double or 10x. For example GPT 3.5 Turbo is 175B parameters.

7.4 SFT Training Success

We see our SFT trained model perform reasonably. We know from our loss curves that further training could continue improving performance. It would be interesting to explore further SFT training of our model with LLM Judges as a criteria to ensure its only learning the best answers, and not just the answers in a static training dataset. But we can safely conclude from these results that our SFT trained model has trained correctly, as we do not expect the Qwen Base model to be able to complete this task meaningfully. A potential experiment here could be to restart SFT training and measure our model's performance on the heldout set, relative to the Qwen Base and Qwen Instruct models, as judged by our LLM judges.

7.5 Judge Agreement and Future Directions

Lastly, we see that both our judges perform similarly. This is a good sign, as they both validate each other's results. However this also opens up the possibility that we could use judges of different calibres to gauge the quality of our candidates in future experiments, to explore if there is more discernment possible between similar models.

8 Conclusion

There is still a lot of room for exploration in how LLMs interact with each other. This idea of LLMs as a judge gives us the framework to do contrastive analyses across models, and explore when they agree or disagree with each other, when they judge each others work as aligned or misaligned with each other. Such efforts could also prove as early signals in other projects, a small example of which would be our own DPO model training project.

More broadly, as LLM benchmarks become more and more common, datasets become more widely shared and re-used, there is an industry wide concern about data integrity. A model could just as well be trained on a publicly available validation dataset, and try to game any benchmarks associated with that dataset. But LLMs as a Judge offer a more dynamic evaluation environment, that is both online and adaptable. As a meta-outcome, we see this as a way to keep moving the frontier on the quality of

LLM training, LLM benchmarks and exploratory work for which LLMs work well with each other versus which which ones do best in a competitive environment.

9 Team Contributions

This is a solo project for Rohit Makhija.

- **Rohit Makhija** explored project ideas for both Custom projects and Default project extensions with several CAs. Rohit implemented the dataloaders, tokenization, loss calculations and trainers for SFT with Qwen over Smoltalk and DPO over UltraFeedback. Rohit developed a pipeline to test multi-agentic behaviors using the Qwen model and CrewAI for multi-agentic implementations. Failing this approach, Rohit implemented other methods to mimic role-play and self-play via the system prompt, only to find out later from the CA team that this approach is unviable since Qwen does not seem to honor instructions in system prompts. Rohit pivoted to the above extension idea after consulting with the CA team.

Changes from Proposal We originally planned to study Multi-agentic behaviors using Crew AI, or by hacking the system prompt. We spent several days on these approaches, only to find out that Qwen ignores system prompts, and does not yet have integration for Crew AI. In the limited time available we pivoted to the above extension idea.

References

Florian E. Dorner, Vivian Y. Nastl, and Moritz Hardt. 2025. Limits to scalable evaluation at the frontier: LLM as Judge won't beat twice the data. arXiv:2410.13341 [cs.LG] `https://arxiv.org/abs/2410.13341`

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A Survey on LLM-as-a-Judge. arXiv:2411.15594 [cs.CL] `https://arxiv.org/abs/2411.15594`

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2025. JudgeBench: A Benchmark for Evaluating LLM-based Judges. arXiv:2410.12784 [cs.AI] `https://arxiv.org/abs/2410.12784`